Sistema de Diagnóstico Acústico de Falhas Mecânicas

Plano de Evolução do Projeto

Diagnóstico da situação atual e recomendações detalhadas para as próximas etapas

Documento	Plano de evolução do projeto
Versão	1.0
Data	Setembro de 2026
Situação analisada	Pastas do projeto e bases de dados em 28 de setembro de 2026
Público	Responsável pelo projeto, equipe de desenvolvimento e equipe que grava os vídeos

Sumário

1. Introdução . . . 3
1.1 Objetivo deste documento . . . 3
1.2 Como ler as recomendações . . . 3

2. Situação atual do projeto . . . 3
2.1 O que o sistema faz . . . 3
2.2 O que foi encontrado . . . 4
2.3 Conclusão . . . 4
2.4 Resumo das recomendações . . . 5

3. Fase 1 — Organizar o projeto . . . 5
3.1 Unificar as duas bases de dados . . . 5
3.2 Remover a instalação duplicada e tirar o programa do OneDrive . . . 6
3.3 Adotar controle de versão (Git) . . . 6
3.4 Fixar as versões das bibliotecas . . . 7

4. Fase 2 — Coletar dados do jeito certo . . . 7
4.1 Criar a classe “Normal” . . . 8
4.2 Não enviar os vídeos pelo WhatsApp como vídeo . . . 8
4.3 Registrar o equipamento de cada gravação . . . 9
4.4 Definir metas de quantidade e um protocolo de gravação . . . 9

5. Fase 3 — Validar antes de confiar nos resultados . . . 10
5.1 Avaliar a IA por equipamento . . . 10
5.2 Verificar o risco de o laudo “entregar” a resposta . . . 11
5.3 Exibir a tabela de erros (matriz de confusão) . . . 11

6. Fase 4 — Funcionalidades para uso por mais pessoas . . . 12
6.1 Juntar backups de vários computadores . . . 12
6.2 Aprovar as confirmações do operador antes do treino . . . 12
6.3 Guardar o histórico de diagnósticos e emitir relatório . . . 13
6.4 Proteger a aba Admin com senha . . . 13

7. Fase 5 — Melhorar o modelo e o código . . . 13
7.1 Considerar a variação do som ao longo do tempo . . . 14
7.2 Testar modelos de áudio pré-treinados . . . 14
7.3 Separar o app.py em módulos . . . 14
7.4 Criar testes automáticos . . . 15

8. Plano de ação sugerido . . . 15
8.1 Sequência recomendada . . . 15
8.2 Se for escolher uma única ação . . . 15
8.3 Checklist dos próximos passos . . . 16

Glossário . . . 16

1. Introdução

1.1 Objetivo deste documento

Este documento reúne, em forma de lista, todas as recomendações para a continuidade do desenvolvimento do Sistema de Diagnóstico Acústico de Falhas Mecânicas. Ele parte de uma análise da situação real do projeto (arquivos, instalações e bases de dados existentes) e explica, para cada recomendação, o problema que ela resolve, por que ela é importante e como colocá-la em prática.

1.2 Como ler as recomendações

Cada recomendação segue a mesma estrutura:

- **Situação atual:** o que acontece hoje no projeto.
- **Por que é importante:** o risco ou a limitação que a recomendação elimina.
- **Como fazer:** as ações concretas, em ordem.
- **Resultado esperado:** o que muda depois que a recomendação for aplicada.

Logo abaixo do título de cada recomendação há uma ficha com três informações: a **prioridade** (alta, média ou baixa), o **esforço** estimado (baixo, médio ou alto) e o **responsável**:

Responsável	Quem é
Desenvolvimento	Quem mantém o software: o responsável pelo projeto, com apoio do assistente de programação (Claude).
Equipe de campo	Quem grava os vídeos dos equipamentos e escreve os laudos.
Usuário	Quem opera o sistema no computador (administrador da base ou operador).

Os termos técnicos utilizados estão explicados no glossário, ao final do documento.

2. Situação atual do projeto

2.1 O que o sistema faz

O sistema recebe um vídeo do equipamento em funcionamento e o laudo escrito pelo técnico. Ele extrai o som do vídeo, remove a voz humana quando ela está presente, calcula 56 características acústicas e as combina com o conteúdo do laudo. Com esses dados, um modelo de inteligência artificial (LightGBM) aprende a reconhecer cada tipo de falha. Na aba **Alimentação**, a IA é ensinada com casos cujo defeito já é conhecido; na aba **Diagnóstico**, ela sugere o problema mais provável de um vídeo novo, e a confirmação do operador passa a ser mais um exemplo de treino.

2.2 O que foi encontrado

Constatação	Detalhe
Dados divididos em duas bases	A instalação principal tem 1 caso. Uma segunda instalação, criada ao extrair o pacote .zip dentro da pasta do projeto (pasta DiagnosticoAcustico_instalador, em 21/09), tem outros 6 casos. São 7 registros, mas apenas 6 vídeos distintos: o arquivo “video 1.mp4” foi cadastrado nas duas bases.
Um único problema cadastrado	Todos os casos têm o rótulo “eletrobomba”.
IA ainda não treinada	O primeiro treinamento exige pelo menos 2 problemas diferentes com 10 vídeos cada. Com um único problema, não há o que comparar.
Vídeos recebidos pelo WhatsApp	Cinco dos seis vídeos são arquivos do WhatsApp, que comprime o áudio.
Gravações quase simultâneas	Os cinco vídeos do WhatsApp foram gravados em 17/09, entre 14h00min07s e 14h00min12s, provavelmente na mesma máquina e na mesma sessão.
Programa duplicado no OneDrive	A segunda instalação contém o próprio ambiente Python (.venv), de 1,6 GB, dentro de uma pasta sincronizada pelo OneDrive.
Sem controle de versão	O projeto não utiliza Git: não há histórico das alterações nem uma forma segura de voltar a uma versão anterior.
Versões de bibliotecas não fixadas	Uma instalação nova já baixa versões diferentes das do computador principal (por exemplo, PyTorch 2.9 em vez de 2.14).

2.3 Conclusão

O software está funcional e testado: extração de áudio, remoção de voz, cálculo das características, treinamento, diagnóstico e instalação em outros computadores foram verificados. **O limite do projeto hoje é a quantidade e a qualidade dos dados, e não o código.** Por isso, as recomendações mais importantes deste documento tratam da forma de coletar os vídeos e de medir, com honestidade, o quanto a IA realmente acerta.

2.4 Resumo das recomendações

Seção	Recomendação	Prioridade	Esforço	Responsável
3.1	Unificar as duas bases de dados	Alta	Baixo	Desenvolvimento
3.2	Remover a instalação duplicada e tirar o programa do OneDrive	Alta	Baixo	Usuário
3.3	Adotar controle de versão (Git)	Alta	Baixo	Desenvolvimento
3.4	Fixar as versões das bibliotecas	Média	Baixo	Desenvolvimento
4.1	Criar a classe “Normal”	Alta	Médio	Equipe de campo
4.2	Não enviar os vídeos pelo WhatsApp como vídeo	Alta	Baixo	Equipe de campo
4.3	Registrar o equipamento de cada gravação	Alta	Baixo	Desenvolvimento e equipe de campo
4.4	Definir metas de quantidade e um protocolo de gravação	Alta	Contínuo	Equipe de campo
5.1	Avaliar a IA por equipamento	Alta	Médio	Desenvolvimento
5.2	Verificar o risco de o laudo “entregar” a resposta	Alta	Médio	Desenvolvimento
5.3	Exibir a tabela de erros (matriz de confusão)	Média	Baixo	Desenvolvimento
6.1	Juntar backups de vários computadores	Média	Médio	Desenvolvimento
6.2	Aprovar as confirmações do operador antes do treino	Média	Médio	Desenvolvimento
6.3	Guardar o histórico de diagnósticos e emitir relatório	Média	Médio	Desenvolvimento
6.4	Proteger a aba Admin com senha	Baixa	Baixo	Desenvolvimento
7.1	Considerar a variação do som ao longo do tempo	Média	Médio	Desenvolvimento
7.2	Testar modelos de áudio pré-treinados	Baixa	Alto	Desenvolvimento
7.3	Separar o app.py em módulos	Média	Médio	Desenvolvimento
7.4	Criar testes automáticos	Média	Médio	Desenvolvimento

3. Fase 1 — Organizar o projeto

Esta fase é rápida e deve ser feita primeiro. Ela evita que dados se percam ou se espalhem e cria uma base segura para as alterações das fases seguintes.

3.1 Unificar as duas bases de dados

Prioridade: alta · Esforço: baixo · Responsável: desenvolvimento

Situação atual. os casos cadastrados estão divididos entre a instalação principal (1 caso) e a instalação criada dentro da pasta DiagnosticoAcustico_instalador (6 casos). Cada instalação tem o seu próprio dataset_master.csv e as suas próprias pastas de áudios e laudos.

Por que é importante. a IA aprende apenas com a base da instalação que está sendo usada. Com os dados divididos, nenhuma das bases chega ao mínimo necessário para treinar, não fica claro qual delas é a oficial, e um backup feito em uma delas não contém os casos da outra. Quanto mais tempo

as duas forem usadas em paralelo, mais difícil fica juntá-las.

Como fazer:

1. Escolher uma única instalação como oficial (ver também a recomendação 3.2).
2. Gerar um backup de cada base pelo botão “Baixar backup (.zip)”, por segurança.
3. Importar para a base oficial os casos da outra base. O sistema já grava a impressão digital (hash SHA-256) de cada vídeo, o que permite identificar e ignorar automaticamente os vídeos repetidos, como o “video 1.mp4”.
4. Conferir, no painel “Base de conhecimento”, se o total ficou correto (6 casos distintos).
5. A partir daí, cadastrar novos casos somente na instalação oficial.

Resultado esperado. uma única base, com todos os casos, sem duplicidades e com backup completo.

3.2 Remover a instalação duplicada e tirar o programa do OneDrive

Prioridade: alta · Esforço: baixo · Responsável: usuário

Situação atual. a segunda instalação tem um ambiente Python (.venv) de 1,6 GB dentro de uma pasta sincronizada pelo OneDrive. A instalação principal também está no OneDrive, na Área de Trabalho.

Por que é importante. o OneDrive tenta enviar para a nuvem milhares de arquivos de bibliotecas que não precisam de cópia, o que consome espaço, internet e processamento. Além disso, a sincronização pode bloquear um arquivo no momento em que o sistema tenta gravá-lo (por exemplo, o dataset_master.csv), causando erros de gravação. O ambiente .venv também não funciona se for copiado para outro computador, portanto sincronizá-lo não serve como backup.

Como fazer:

1. Depois de unificar as bases (3.1), apagar a pasta DiagnosticoAcustico_instalador.
2. Instalar a versão oficial em uma pasta local, fora do OneDrive, por exemplo C:\DiagnosticoAcustico, levando os dados pelo backup (.zip).
3. Usar o OneDrive apenas para guardar os arquivos de backup gerados pelo sistema.

Resultado esperado. menos sincronização desnecessária, fim dos riscos de bloqueio de arquivo e uma regra clara: o programa fica no computador; o backup fica na nuvem.

3.3 Adotar controle de versão (Git)

Prioridade: alta · Esforço: baixo · Responsável: desenvolvimento

Situação atual. o código não tem histórico. As versões anteriores existem apenas como cópias manuais de pastas.

Por que é importante. o Git registra cada alteração do código com data e descrição. Isso permite voltar a uma versão que funcionava caso uma mudança cause problemas, saber exatamente o que mudou entre duas versões, identificar qual versão está instalada em cada computador e trabalhar em conjunto com outras pessoas sem que uma sobrescreva o trabalho da outra. É a prática padrão em qualquer projeto de software.

Como fazer:

1. Criar o repositório Git na pasta do projeto.
2. Configurar o arquivo .gitignore para **não** versionar o ambiente .venv, os pacotes .zip e os dados (áudios, laudos, dataset_master.csv e modelos). Os dados são grandes e podem ser confidenciais; eles continuam protegidos pelo backup do próprio sistema.
3. Publicar o repositório em um serviço privado (GitHub ou GitLab), que funciona também como cópia de segurança do código.
4. Registrar cada alteração (commit) com uma descrição clara e marcar com uma etiqueta (tag) cada versão enviada para outros computadores, por exemplo v1.0 e v1.1.

Resultado esperado. todo o histórico do código preservado, possibilidade de desfazer alterações com segurança e controle sobre qual versão está em uso em cada lugar.

3.4 Fixar as versões das bibliotecas

Prioridade: média · Esforço: baixo · Responsável: desenvolvimento

Situação atual. o arquivo requirements.txt define apenas versões mínimas (por exemplo, torch>=2.0). Cada instalação nova baixa as versões mais recentes disponíveis naquele dia. Isso já acontece hoje: uma instalação feita em setembro recebeu o PyTorch 2.9, enquanto o computador principal usa o 2.14.

Por que é importante. computadores com versões diferentes podem se comportar de forma diferente, e uma versão futura de alguma biblioteca pode simplesmente quebrar o programa. Há ainda um risco específico: o modelo treinado é salvo em um formato que depende das versões do scikit-learn e do LightGBM. Um backup levado para um computador com versões diferentes pode não conseguir carregar o modelo, exigindo um novo treinamento.

Como fazer:

1. Em uma instalação validada, gerar a lista exata de versões (comando pip freeze).
2. Usar essa lista como requirements.txt oficial, distribuído com o pacote de instalação.
3. Atualizar as bibliotecas somente de forma planejada: testar a nova versão, confirmar que tudo funciona e só então atualizar a lista.

Resultado esperado. todos os computadores com o mesmo comportamento e instalações reproduzíveis, hoje e no futuro.

4. Fase 2 — Coletar dados do jeito certo

Esta é a fase mais importante do projeto. A qualidade da IA depende muito mais da qualidade e da variedade dos exemplos do que de qualquer ajuste no código. Uma IA treinada com dados ruins aprende padrões errados, e nenhuma técnica corrige isso depois.

4.1 Criar a classe “Normal”

Prioridade: alta · Esforço: médio · Responsável: equipe de campo

Situação atual. só existem gravações de equipamentos com defeito. Não há exemplos de equipamentos funcionando corretamente.

Por que é importante. o classificador sempre divide 100% da probabilidade entre os problemas que conhece. Sem uma classe “Normal”, um equipamento em perfeito estado será obrigatoriamente apontado como portador de algum defeito. Na prática, a primeira pergunta de quem usa o sistema costuma ser justamente “este equipamento tem algum problema?”, e sem essa classe a IA não consegue responder.

Como fazer:

- Gravar equipamentos em bom estado, dos mesmos modelos e nas mesmas condições em que são gravados os equipamentos com defeito (mesmo local, mesma distância, mesmo tipo de celular).
- Cadastrar esses vídeos sempre com o rótulo exato “Normal”.
- Manter a quantidade de vídeos da classe Normal pelo menos igual à do problema mais frequente.

Resultado esperado. a IA passa a distinguir “sem defeito” de “com defeito”, e não apenas um defeito de outro.

4.2 Não enviar os vídeos pelo WhatsApp como vídeo

Prioridade: alta · Esforço: baixo · Responsável: equipe de campo

Situação atual. cinco dos seis vídeos cadastrados foram recebidos pelo WhatsApp.

Por que é importante. ao enviar um vídeo como “vídeo”, o WhatsApp o recomprime para economizar dados, reduzindo a qualidade da imagem e também do áudio. O áudio é exatamente a informação que a IA analisa: detalhes finos do som, como componentes agudos típicos de alguns defeitos, podem ser atenuados ou perdidos. Além disso, a compressão varia conforme o celular e a versão do aplicativo, criando diferenças artificiais entre as gravações que a IA pode acabar aprendendo no lugar do defeito real.

Como fazer:

- No WhatsApp, enviar o arquivo como **Documento** (e não como vídeo): assim o arquivo original é preservado.
- Alternativamente, copiar o arquivo do celular por cabo USB ou enviá-lo pelo OneDrive, Google Drive ou e-mail, que não alteram o arquivo.
- Guardar os arquivos originais em uma pasta de arquivo, separada do sistema.
- Sempre que possível, regravar os casos atuais e enviá-los da forma correta.

Resultado esperado. a IA recebe o som como ele realmente foi captado, sem perdas nem distorções causadas pelo envio.

4.3 Registrar o equipamento de cada gravação

Prioridade: alta · Esforço: baixo · Responsável: desenvolvimento e equipe de campo

Situação atual. o sistema não guarda de qual equipamento veio cada vídeo. Os nomes de arquivo do WhatsApp ("WhatsApp Video 2026-09-17 at 14.00.07 (1).mp4") não ajudam a identificá-lo.

Por que é importante. sem essa informação, não é possível saber se um problema está representado por várias máquinas ou por uma só, nem avaliar a IA de forma honesta (ver 5.1). Também fica impossível rastrear a origem de um caso suspeito.

Como fazer:

- **Desenvolvimento:** acrescentar ao formulário de cadastro um campo "Equipamento" (modelo e número de série ou de patrimônio) e, opcionalmente, um campo "Condições da gravação" (carga, rotação, local e celular utilizado). Esses dados devem ficar em colunas próprias do dataset_master.csv, e **não** no texto do laudo, para não interferirem na análise (ver 5.2).
- **Equipe de campo, até o campo existir:** renomear cada arquivo antes do envio, seguindo um padrão como EQUIPAMENTO_DATA_SEQUÊNCIA (por exemplo, EB-0123_2026-09-17_01.mp4). O sistema guarda o nome original do arquivo, então essa informação não se perde.

Resultado esperado. cada caso ligado ao seu equipamento, o que permite a avaliação por equipamento e o controle da diversidade dos dados.

4.4 Definir metas de quantidade e um protocolo de gravação

Prioridade: alta · Esforço: contínuo · Responsável: equipe de campo

Situação atual. há 6 vídeos de um único problema, gravados em uma única sessão.

Por que é importante. o sistema exige no mínimo 10 vídeos por problema para incluí-lo no treinamento, mas esse é apenas um mínimo operacional. Para que a IA aprenda o defeito, e não as particularidades de uma máquina ou de um dia, são necessários exemplos variados e gravados sempre da mesma maneira.

Metas sugeridas:

Etapa	Meta
Primeiro treinamento	2 problemas mais a classe Normal, com pelo menos 10 vídeos cada (30 vídeos no total)
Avaliação confiável	30 ou mais vídeos por problema, vindos de pelo menos 3 equipamentos diferentes
Equilíbrio	Quantidades semelhantes entre os problemas; nenhum problema com mais que o dobro de outro

Protocolo de gravação sugerido:

- ☐ Equipamento em funcionamento estável (sem ligar ou desligar durante a gravação).
- ☐ Celular sempre na mesma posição e à mesma distância do componente, definida pela equipe (por exemplo, 30 cm).
- ☐ Duração entre 30 segundos e 3 minutos.
- ☐ Ninguém falando durante a gravação; evitar vento e outras máquinas ruidosas por perto.
- ☐ Microfone do celular desobstruído (sem capa ou mão cobrindo).
- ☐ Um único defeito por gravação, confirmado por um técnico.
- ☐ Laudo descrevendo o que foi observado e anotação do equipamento e das condições (ver 4.3).
- ☐ Envio do arquivo original (ver 4.2).

Resultado esperado. dados suficientes e comparáveis entre si, condição necessária para que a IA generalize para equipamentos que ela nunca viu.

5. Fase 3 — Validar antes de confiar nos resultados

Depois do primeiro treinamento, o sistema exibe uma "acurácia estimada". Esta fase garante que esse número reflita o desempenho real da IA em campo, e não uma ilusão criada pela forma como os dados foram coletados.

5.1 Avaliar a IA por equipamento

Prioridade: alta · Esforço: médio · Responsável: desenvolvimento

Situação atual. a acurácia é calculada por validação cruzada: o sistema separa parte dos casos, treina com os demais e confere os acertos nos casos separados, repetindo o processo até 5 vezes. A separação é feita ao acaso, sem considerar de qual equipamento veio cada vídeo.

Por que é importante. vídeos da mesma máquina, gravados no mesmo momento (como os cinco vídeos do WhatsApp de 17/09), são quase idênticos. Se um deles fica no treino e outro no teste, a IA "acerta" por reconhecer aquela gravação, e não o defeito. É como estudar com as respostas da prova: a nota sai alta, mas não mede o aprendizado. O resultado é uma acurácia otimista, que cai quando a IA encontra um equipamento novo.

Como fazer:

- Com o campo "Equipamento" disponível (4.3), mudar a validação para que todos os vídeos de um mesmo equipamento fiquem sempre juntos: ou todos no treino, ou todos no teste (técnica conhecida como validação agrupada, ou "deixar um equipamento de fora").
- Exibir no painel as duas medidas: a acurácia atual e a acurácia por equipamento. A segunda é a que indica o desempenho esperado em campo.

Resultado esperado. um número de acurácia confiável, que responde à pergunta certa: "quanto a IA acerta em um equipamento que ela nunca viu?".

5.2 Verificar o risco de o laudo "entregar" a resposta

Prioridade: alta · Esforço: médio · Responsável: desenvolvimento

Situação atual. a IA recebe, além do som, uma representação numérica do texto do laudo. Os dois são analisados juntos.

Por que é importante. se os laudos de um problema sempre mencionam o nome do defeito (por exemplo, "rolamento com folga"), a IA pode aprender a ler a resposta no texto em vez de ouvir o som. O resultado aparenta ser excelente, mas depende de alguém já saber qual é o defeito. Na aba Diagnóstico, o laudo também é escrito pelo operador; nesse cenário, a IA apenas repetiria a suspeita do operador, sem acrescentar nada do áudio.

Como fazer:

1. Medir a acurácia em três versões do modelo: somente com o áudio, somente com o texto e com áudio mais texto (a versão atual).
2. Interpretar os resultados conforme a tabela abaixo.
3. Orientar a equipe a escrever laudos que descrevam **sintomas observados** ("ruído agudo intermitente perto do mancal") e não o diagnóstico ("rolamento com defeito").

Resultado da comparação	Significado	Ação
Áudio + texto parecido com somente áudio	O som é quem sustenta o diagnóstico	Situação saudável; manter
Somente texto parecido com áudio + texto, e somente áudio bem pior	O texto domina: a IA está lendo a resposta no laudo	Revisar a redação dos laudos, reduzir o peso do texto ou torná-lo opcional
Áudio + texto melhor que cada um isolado	Os dois se complementam	Situação ideal; manter a orientação sobre a redação dos laudos

Resultado esperado. certeza de que a IA diagnostica pelo som do equipamento, que é o objetivo do sistema.

5.3 Exibir a tabela de erros (matriz de confusão)

Prioridade: média · Esforço: baixo · Responsável: desenvolvimento

Situação atual. o painel mostra apenas uma acurácia geral.

Por que é importante. uma acurácia de 80% pode esconder situações muito diferentes: a IA pode acertar sempre um problema e errar muito outro. A matriz de confusão é uma tabela que cruza o problema real (linhas) com o problema apontado pela IA (colunas) e mostra exatamente quais problemas estão sendo confundidos entre si.

Exemplo ilustrativo (valores fictícios):

Problema real \ apontado pela IA	Normal	Rolamento	Desbalanceamento
Normal	18	1	1
Rolamento	2	15	3
Desbalanceamento	0	4	16

Nesse exemplo, a IA confunde com frequência Rolamento e Desbalanceamento, o que indica que esses dois problemas precisam de mais exemplos ou de gravações mais focadas no componente defeituoso.

Como fazer. calcular a matriz a partir das previsões da validação (preferencialmente a validação por equipamento, 5.1) e exibi-la no painel "Base de conhecimento" após cada treinamento, junto com a taxa de acerto de cada problema.

Resultado esperado. visão clara dos pontos fracos da IA e orientação objetiva sobre quais gravações coletar em seguida.

6. Fase 4 — Funcionalidades para uso por mais pessoas

Estas recomendações se tornam importantes quando o sistema passar a ser usado em mais de um computador ou por mais de uma pessoa.

6.1 Juntar backups de vários computadores

Prioridade: média · Esforço: médio · Responsável: desenvolvimento

Situação atual. cada computador tem a sua própria base. A restauração de um backup **substitui** os dados do computador de destino; não existe uma forma de somar os casos de duas bases.

Por que é importante. com várias pessoas coletando dados em computadores diferentes, as bases ficam fragmentadas, exatamente como aconteceu com as duas instalações atuais. Copiar manualmente o dataset_master.csv de um computador para outro apaga os casos do computador de destino.

Como fazer. criar a função "Importar backup (mesclar)": o sistema lê o arquivo de backup, acrescenta à base apenas os casos que ainda não existem (identificados pela impressão digital SHA-256 de cada vídeo, que já é gravada), copia os áudios e laudos correspondentes, informa quantos casos foram adicionados e ignorados e retreina a IA.

Resultado esperado. uma base central que reúne, com segurança, o trabalho de todos os computadores.

6.2 Aprovar as confirmações do operador antes do treino

Prioridade: média · Esforço: médio · Responsável: desenvolvimento

Situação atual. na aba Diagnóstico, um clique do operador salva o caso imediatamente e retreina a IA.

Por que é importante. uma confirmação errada, por engano ou por falta de verificação, ensina a IA de forma errada, e o erro se propaga para os diagnósticos seguintes. Quanto mais pessoas usarem o sistema, maior esse risco.

Como fazer:

- Enviar as confirmações do operador para uma fila de "casos pendentes", em vez de incluí-las diretamente no treinamento.

- Permitir que o responsável técnico ouça o áudio, leia o laudo e aprove ou rejeite cada caso pendente.
- Incluir no treinamento apenas os casos aprovados e registrar quem confirmou e quem aprovou.

Resultado esperado. o aprendizado contínuo da IA mantido, com controle de qualidade sobre o que ela aprende.

6.3 Guardar o histórico de diagnósticos e emitir relatório

Prioridade: média · Esforço: médio · Responsável: desenvolvimento

Situação atual. um diagnóstico que não é confirmado não deixa registro. Também não fica guardado o que a IA sugeriu em cada análise.

Por que é importante. o histórico é necessário para a rastreabilidade da manutenção e permite calcular a medida mais honesta de todas: a taxa de acerto da IA em uso real, comparando a sugestão feita com o defeito confirmado depois pela inspeção. Um relatório em PDF também facilita anexar o diagnóstico às ordens de serviço.

Como fazer. registrar cada diagnóstico (data, arquivo, laudo, probabilidades, decisão final e operador), exibir o histórico em uma nova área do sistema e oferecer a exportação de um relatório em PDF por diagnóstico.

Resultado esperado. rastreabilidade completa e acompanhamento do desempenho real da IA ao longo do tempo.

6.4 Proteger a aba Admin com senha

Prioridade: baixa (média, se houver vários usuários) · Esforço: baixo · Responsável: desenvolvimento

Situação atual. qualquer pessoa com acesso ao computador pode cadastrar e excluir casos na aba Alimentação (Admin).

Por que é importante. com vários usuários, aumenta o risco de exclusões acidentais ou de cadastros incorretos na base, que é o principal patrimônio do projeto.

Como fazer. criar dois perfis: o operador acessa apenas a aba Diagnóstico, e o administrador, mediante senha, acessa também a Alimentação e a gestão de casos. A senha deve ser guardada de forma cifrada, fora do código. Trata-se de um controle de uso; a segurança principal continua sendo a conta do Windows.

Resultado esperado. somente pessoas autorizadas alteram a base de conhecimento.

7. Fase 5 — Melhorar o modelo e o código

As melhorias de modelo só devem ser feitas depois que houver dados suficientes (Fase 2) e uma forma confiável de medir o resultado (Fase 3). Sem isso, não há como saber se uma mudança realmente melhorou a IA. As melhorias de código podem ser feitas a qualquer momento.

7.1 Considerar a variação do som ao longo do tempo

Prioridade: média (após a Fase 3) · Esforço: médio · Responsável: desenvolvimento

Situação atual. o áudio é dividido em janelas de 2 segundos, e as 56 características de todas as janelas são resumidas pela média.

Por que é importante. a média dilui eventos curtos. Uma batida que aparece a cada poucos segundos, típica de alguns defeitos, quase desaparece quando é somada a longos trechos normais. Informações como "o som varia muito" ou "há picos esporádicos" também se perdem.

Como fazer. além da média, registrar medidas de variação entre as janelas, como desvio-padrão, valor máximo e percentis. Outra opção é classificar cada janela separadamente e combinar os resultados por votação. Como os áudios processados ficam guardados, é possível recalcular as características de todos os casos já cadastrados, sem precisar regravá-los.

Resultado esperado. melhor detecção de defeitos intermitentes, confirmada pela avaliação por equipamento.

7.2 Testar modelos de áudio pré-treinados

Prioridade: baixa por enquanto · Esforço: alto · Responsável: desenvolvimento

Situação atual. as características do som são calculadas por fórmulas definidas manualmente.

Por que é importante. existem redes neurais treinadas com milhões de sons (por exemplo, PANNs e BEATs) que convertem um áudio em um conjunto de números que o descreve de forma muito rica. Esses modelos costumam funcionar bem mesmo com poucos exemplos por classe. Em contrapartida, são mais pesados e aumentam o tempo de processamento no computador.

Como fazer. somente depois das Fases 2 e 3, comparar o modelo atual com uma versão que utilize um desses modelos, sempre pela avaliação por equipamento. Adotar a mudança apenas se o ganho for claro.

Resultado esperado. possível ganho de acurácia, decidido com base em medições, e não em suposições.

7.3 Separar o app.py em módulos

Prioridade: média · Esforço: médio · Responsável: desenvolvimento

Situação atual. todo o sistema está em um único arquivo com cerca de 1.700 linhas, que inclui também cerca de 57 KB de imagens (logotipos) embutidas no código.

Por que é importante. um arquivo grande é mais difícil de entender e de alterar sem efeitos colaterais, e dificulta a criação de testes (7.4) e o reaproveitamento de partes do sistema, como um processamento em lote sem interface.

Como fazer. separar o código em arquivos por responsabilidade: extração de áudio e remoção de voz, cálculo das características, gestão dos dados, treinamento e avaliação, e interface (uma parte para cada aba). Os logotipos e o estilo visual passam a ser arquivos próprios. O comportamento do sistema não muda.

Resultado esperado. código mais organizado, fácil de manter e de testar.

7.4 Criar testes automáticos

Prioridade: média · Esforço: médio · Responsável: desenvolvimento

Situação atual. durante o desenvolvimento, o sistema foi verificado com testes feitos manualmente: vídeo sem áudio, vídeo com fala, vídeos duplicados, normalização de rótulos, treinamento, exclusão de casos e a equivalência exata das características calculadas. Esses testes não ficaram registrados de forma reutilizável.

Por que é importante. testes automáticos verificam em poucos minutos, antes de cada nova versão, que nada deixou de funcionar. Um deles é especialmente importante: confirmar que as características do som continuam sendo calculadas exatamente da mesma forma. Se esse cálculo mudar sem intenção, a IA passa a comparar dados incompatíveis com os que aprendeu.

Como fazer. transformar as verificações já realizadas em uma suíte de testes (pytest) que usa áudios sintéticos, sem depender de dados reais, e executá-la antes de cada versão marcada no Git (3.3).

Resultado esperado. novas versões lançadas com segurança e erros detectados antes de chegarem aos usuários.

8. Plano de ação sugerido

8.1 Sequência recomendada

Etapa	Quando	O que fazer
Fase 1 — Organizar	Imediatamente (poucos dias)	Unificar as bases, remover a instalação duplicada, adotar o Git e fixar as versões (3.1 a 3.4).
Fase 2 — Coletar	A partir de agora, de forma contínua	Gravar a classe Normal e um segundo problema, seguindo o protocolo e enviando os arquivos originais (4.1 a 4.4).
Preparação da Fase 3	Em paralelo à coleta	Implementar o campo Equipamento, a avaliação por equipamento, a comparação áudio × texto e a matriz de confusão (4.3 e 5.1 a 5.3).
Fase 3 — Validar	Quando houver 2 problemas e a classe Normal com 10 vídeos cada	Analisar os resultados e ajustar a coleta e a redação dos laudos.
Fase 4 — Ampliar o uso	Antes de levar o sistema a mais usuários	Mesclar backups, fila de aprovação, histórico e senha (6.1 a 6.4).
Fase 5 — Aperfeiçoar	Com cerca de 30 vídeos por problema	Melhorias do modelo (7.1 e 7.2); organização e testes do código a qualquer momento (7.3 e 7.4).

8.2 Se for escolher uma única ação

Unificar as bases e começar a gravar a classe **Normal** e um **segundo problema**, com os arquivos originais em vez de vídeos do WhatsApp. Essa ação sozinha destrava o primeiro treinamento da IA. Enquanto os dados são coletados, a parte técnica (fusão das bases, Git, campo Equipamento, avaliação por equipamento e comparação áudio × texto) pode ser preparada, de modo que, quando os vídeos chegarem, já exista uma medida confiável do quanto a IA acerta.

8.3 Checklist dos próximos passos

- ☐ Unificar as duas bases de dados (3.1).
- ☐ Apagar a pasta DiagnosticoAcustico_instalador e mover o programa para fora do OneDrive (3.2).
- ☐ Criar o repositório Git privado (3.3).
- ☐ Fixar as versões das bibliotecas no requirements.txt (3.4).
- ☐ Combinar com a equipe de campo o protocolo de gravação e a forma de envio dos arquivos (4.2 e 4.4).
- ☐ Gravar pelo menos 10 vídeos da classe Normal (4.1).
- ☐ Gravar pelo menos 10 vídeos de um segundo problema (4.4).
- ☐ Implementar o campo Equipamento (4.3).
- ☐ Implementar a avaliação por equipamento, a comparação áudio × texto e a matriz de confusão (5.1 a 5.3).

Glossário

Termo	Significado
Acurácia	Proporção de acertos da IA. Uma acurácia de 90% significa que, a cada 10 casos avaliados, a IA acertou 9.
Backup	Cópia de segurança dos dados. No sistema, é o arquivo .zip gerado pelo botão “Baixar backup”.
Classe (ou rótulo)	Cada problema que a IA aprende a reconhecer, como “Rolamento” ou “Normal”.
Commit	Registro de uma alteração no Git, com data, autor e descrição.
Git	Sistema de controle de versão que guarda o histórico das alterações do código.
Hash SHA-256 (impressão digital)	Código calculado a partir do conteúdo de um arquivo. Arquivos idênticos têm o mesmo código, o que permite detectar vídeos repetidos.
LightGBM	Algoritmo de aprendizado de máquina baseado em árvores de decisão, usado pelo sistema para classificar os problemas.
Matriz de confusão	Tabela que cruza o problema real com o problema apontado pela IA e mostra quais problemas são confundidos entre si.
Modelo pré-treinado	Rede neural já treinada com grandes volumes de dados, reaproveitada para uma nova tarefa.
Tag (etiqueta)	Marcação de uma versão específica do código no Git, por exemplo v1.0.
Validação agrupada (por equipamento)	Forma de avaliação em que todos os vídeos de um equipamento ficam juntos, ou no treino ou no teste, simulando um equipamento que a IA nunca viu.
Validação cruzada	Forma de avaliação em que parte dos casos é separada para teste, a IA é treinada com o restante e o processo se repete várias vezes.
.venv (ambiente virtual)	Pasta com o Python e as bibliotecas usadas pelo sistema, criada pelo iniciar.bat em cada computador.